

Курс «Комбинаторика и
теория графов»
Лекция 1. Теория графов.
Введение.



ПОЗНАКОМИМСЯ

Горденко Мария Константиновна

- Ведущий эксперт Института статистических исследований и экономики знаний НИУ ВШЭ
- Старший преподаватель департамента программной инженерии ФКН НИУ ВШЭ (с 2017 г.), НИТУ МИСИС (с 2021 г.)
- Associate Engineer (с 2019 г.)
- Лектор Российского общества «Знание» (с 2021 г.)

Награды

- Лучший преподаватель НИУ ВШЭ - 2022
- Победитель 1 сезона всероссийского конкурса «Лига Лекторов» Российского общества «Знание» (топ-50 лучших лекторов страны)
- Победитель межрегионального конкурса «Лучший молодой преподаватель» (Ростов-на-Дону, 2021)
- Лауреат Всероссийского конкурса инновационных образовательных технологий «Лучший молодой преподаватель 20.21» (Москва, 2021)

Контакты

- Личная страница - <https://www.hse.ru/staff/gordenko>
- Почта – mkgordenko@gmail.com
- VK – https://vk.com/m_gordenko

Коммуникации

- <https://t.me/+w8ma-etJ1VY1ZjAy>





ПРЕПОДАВАЕМЫЕ ДИСЦИПЛИНЫ

- 1) Алгоритмы и структуры данных
 - 2) Введение в программирование
 - 3) Групповая динамика и коммуникации в профессиональной практике программной инженерии
 - 4) Дискретная математика
 - 5) Подготовка к профессиональной вневендорной сертификации (English, IEEE)
 - 6) Программирование
- 



ТЕМАТИЧЕСКОЕ НАПОЛНЕНИЕ КУРСА

1 часть. Теория сложности (1 лекция)

2 часть. Теория графов

3 часть. Комбинаторика





2 ЧАСТЬ. ТЕОРИЯ ГРАФОВ

- 1) Основы теории графов. Представления графов в памяти компьютера. Дерево. Бинарное дерево. Бинарное дерево поиска. Структуры set, map, очередь с приоритетами. Красно-черное дерево, multiset, multimap. Расстояния: центры, медианы, эксцентриситет. Подграфы. Изоморфизм. Операции (разрез, удаление). Связность графов. Сильная и слабая связность. Компоненты связности.
 - 2) Каркасы (остовы) и хорды в связном графе. Обход в глубину (DFS). Обход в ширину (BFS). Циклы. Эйлеров путь/цикл. Гамильтон цикл. Цикл де Брейна. Определение, свойства, операции над циклами. Пространство циклов. Базис циклов. Цикломатическая матрица. Построение базисной цикломатической матрицы.
 - 3) Деревья. Дерево и лес. Характеристические свойства деревьев. Ориентированные, упорядоченные и бинарные деревья.
- 



2 ЧАСТЬ. ТЕОРИЯ ГРАФОВ

- 4) Двудольные и многодольные графы
 - 5) Поиск кратчайших путей в графе. Алгоритм Дейкстры. Алгоритм Флойда-Уоршелла. Алгоритм Белмана-Форда.
 - 6) Минимальное остовное дерево.
 - 7) Потоки в графах. Поток минимальной/максимальной стоимости. Алгоритм Форда-Фалкерсона. Многопоточные сети. Потоки в сетях.
 - 8) Планарные графы. Укладка графа. Плоские графы. Эйлерова характеристика (формула Эйлера). Графы K_5 и $K_{3,3}$. Критерий планарности Понтрягина-Куратовского. Планарный граф. Раскраска графа
- 



3 ЧАСТЬ. КОМБИНАТОРИКА

- 1) Комбинаторные схемы. Определение чисел размещений, перестановок и сочетаний предметов без повторений и с повторениями, определение чисел упорядоченных и неупорядоченных разбиений конечных множеств, бином Ньютона и полиномиальная формула. Принцип Дирихле. Полиномиальная формула. Полиномиальное множество.
- 2) Оценки для факториалов и чисел. Формула Стирлинга. Рекуррентные соотношения в комбинаторике. Разбиение чисел на слагаемые. Задача Эйлера. Числа Фибоначчи. Задача о размножении кроликов. Линейные рекуррентные соотношения с постоянными коэффициентами.
- 3) Порождение комбинаторных конфигураций и их пересчет. Размещения, перестановки, сочетания без повторений и с повторениями. Подстановки, группа подстановок. Генерация перестановок. Биномиальные коэффициенты, треугольник Паскаля, генерация подмножеств. Разбиения. Числа Стирлинга первого и второго рода, число Белла.



ОЦЕНКА



- 1) ИДЗ. Яндекс.Контесты на алгоритмы (по 1-4 задачи)
- 2) Контрольная работа 1 и 2
- 3) Активность на лекциях и семинарах

Зачет с оценкой:

Более 85% баллов - отлично

75-84% балла - хорошо

55-74% балла - удовлетворительно

менее 55% баллов - неудовлетворительно





РАБОТА В Я.КОНТЕСТ

- 1) Логины и пароли сгенерированы и будут высланы
- 2) В рамках данной дисциплины сознательно не делается акцент на декомпозиции задачи и программы
- 3) Контест позволяет ускорить и автоматизировать проверку. У студента есть несколько попыток. Результаты тестирования задачи появляются практически мгновенно. Контест позволяет провести проверку задачи на плагиат.

ПЕРВЫЙ КОНТЕСТ

1) Заходим по ссылке и вводим полученные логин и пароль

Яндекс Контеcт

Компиляторы Значения ошибок

Авторизация

Авторизуйтесь, используя предоставленные координатором логин и пароль

Логин

misis_dm_algos_20_21_***

Пароль

Войти

ПЕРВЫЙ КОНТЕСТ

Яндекс Контеcт

Русский misis_dm_algos_20_21_

Компиляторы Значения ошибок

DM Algos 2021. #1

🕒 10 фев 2021, 23:11:26

старт: 10 фев 2021, 00:00:00

финиш: 20 фев 2021, 00:00:00

до финиша: 9д.

...

Объявления жюри

Задачи Посылки

Вы участвуете в соревновании

ПЕРВЫЙ КОНТЕСТ

Компиляторы Значения ошибок

DM Algos 2021. #1

🕒 10 фев 2021, 23:11:53

Объявление жюри

старт: 10 фев 2021, 00:00:00

финиш: 20 фев 2021, 00:00:00

до финиша: 9д.

...

Задачи [Посылки](#)

A. DM Algos. A+B

A. DM Algos. A+B

B. DM Algos. Сумма цифр

Ограничение времени	1 секунда
Ограничение памяти	64Mb
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Формат ввода

На первой строке входного файла находятся два целых числа a и b ($-10^9 \leq a, b \leq 10^9$).

Формат вывода

Вашей программе требуется вывести единственное число — сумму заданных чисел $a + b$.

Пример 1

Ввод

2 3

Вывод

5

ПЕРВЫЙ КОНТЕСТ

Язык

C# (MS .Net Core 3.1) + ... ▾

Набрать здесь

Отправить файл

1

Отправить

i

осталось 100 попыток

Следующая

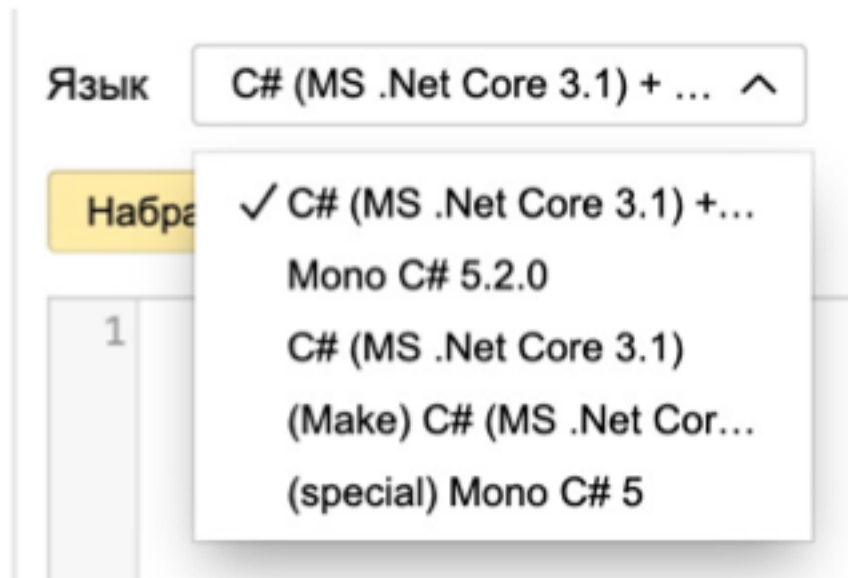
i

Посылок нет

Сообщение	Кратко	Сообщается ли номер теста?	Значение вердикта	Возможная причина
OK	OK	Нет	Решение зачтено	Программа верно работает на соответствующем наборе тестов
Compilation error	CE	Нет	Компиляция программы завершилась с ошибкой	1. в программе допущена синтаксическая или семантическая ошибка 2. неправильно указан язык
Wrong answer	WA	Да	Ответ неверен	1. ошибка в программе 2. неверный алгоритм
Presentation error	PE	Да	Тестирующая система не может проверить выходные данные, так как их формат не соответствует описанному в условиях задачи	1. неверный формат вывода 2. программа не печатает результат 3. лишний вывод
Time-limit exceeded	TL	Да	Программа превысила установленный лимит времени	1. ошибка в программе 2. неэффективное решение
Memory limit exceeded	ML	Да	Программа превысила установленный в условиях лимит памяти	1. ошибка в программе (например, бесконечная рекурсия) 2. неэффективное решение
Output limit exceeded	OL	Да	Программа превысила установленный в условиях лимит вывода	1. программа выводит больше информации, чем установлено в ограничениях
Run-time error	RE	Да	Программа завершила работу с ненулевым кодом возврата	1. ошибка выполнения 2. программа на C или C++ не завершается оператором return 0 3. ненулевой код возврата указан явно 4. Программа на Java описана в пакете
Precompile check failed	PCF	Нет	Программа не прошла проверку на качество кода перед компиляцией	1. плохое качество кода 2. неправильно отформатированный код
Idleness limit exceeded	IL	Да	Программа слишком долго не отвечала на запросы системы и не выполняла действий	1. программа ожидает ввода с консоли, которого не должно быть 2. не использован flush()

ГДЕ ПИСАТЬ КОД?

- 1) Visual Studio и другие IDE
- 2) <https://repl.it/>



Алгоритм

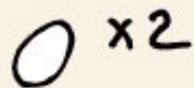
Алгоритм - это некоторая последовательность действий, которые необходимо совершить для достижения нужного результата.

Яичница с зеленым луком и помидорами черри

Chef-daw



помидоры
черри



яйцо



зеленый
лук



оливковое
масло



перец



соль



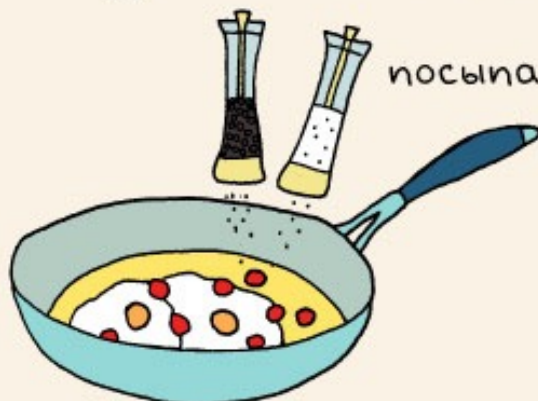
разрезать



обжарить



вылить



посыпать

пожарить



нарезать



ГОТОВО

Свойства алгоритма

- Конечность (результативность)
- Дискретность
- Понятность
- Точность
- Массовость
- Детерминированность (определенность)

2009...



50 miles away...



Сложность алгоритма

- Временная сложность
- Пространственная (емкостная) сложность

В чем измерять временную сложность?

- В секундах?
- В котиках?

В чем измерять временную сложность?

- В секундах?
- В котиках?
- В элементарных операциях!

Модель RAM (Random Access Machine)

- память состоит из ячеек, каждая из которых имеет адрес и может хранить один элемент данных;
- каждое обращение к памяти занимает одну единицу времени, независимо от номера адресуемой ячейки;
- количество памяти достаточно для выполнения любого алгоритма;
- процессор выполняет любую элементарную операцию (основные логические и арифметические операции, чтение из памяти, запись в память, вызов подпрограммы и т.п.) за один временной шаг;
- циклы и функции не считаются элементарными операциями.

Пример – сложение столбиком

$$\begin{array}{r} 1 \\ + 1018 \\ + 1105 \\ \hline 2123 \end{array}$$

Пример – умножение столбиком

$$\begin{array}{r} \times 1011 \\ 1101 \\ \hline 1011 \\ 0000 \\ 1011 \\ 1011 \\ \hline 1113111 \end{array}$$

Пример – посчитать число элементарных операций

```
int min = Integer.MAX_VALUE;
int max = Integer.MIN_VALUE;
for (int x : array) {
    if (x < min) min = x;
    if (x > max) max = x;
}
```

```
int min = Integer.MAX_VALUE;
int max = Integer.MIN_VALUE;
for (int x : array) {
    if (x < min) min = x;
}
for (int x : array) {
    if (x > max) max = x;
}
```

Упрощения

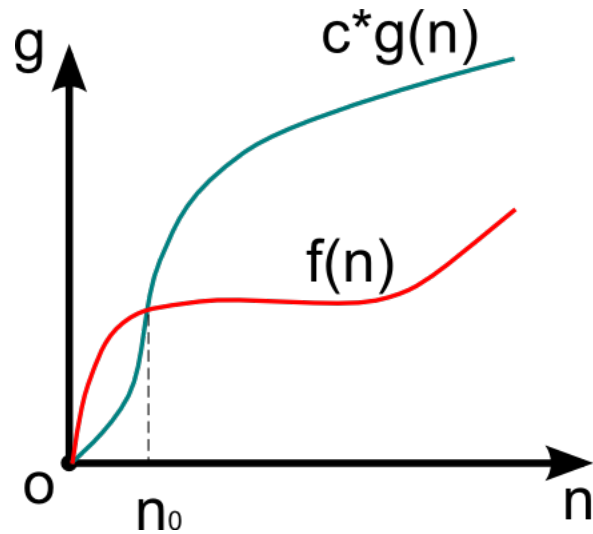
- Оцениваем самую быстрорастущую часть («худшее») слагаемого;
- Игнорируем коэффициенты

Получаем асимптотическую сложность!

Big-O

- Оценка худшего случая

$$f(n) \in O(g(n)) \quad \exists(C > 0), n_0 : \forall(n > n_0) |f(n)| \leq |Cg(n)| \quad \text{или} \\ \exists(C > 0), n_0 : \forall(n > n_0) f(n) \leq Cg(n)$$



Оценка O (O большое)

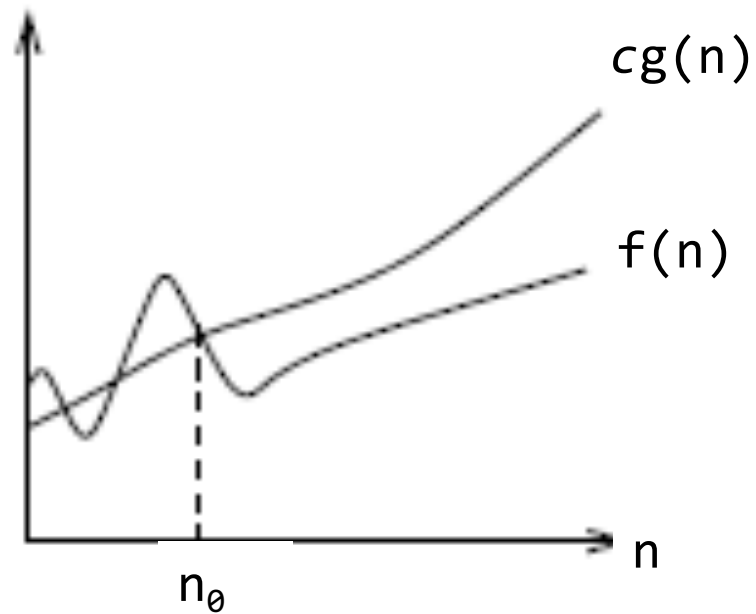
Оценка O требует, чтобы функция $f(n)$ не превышала значения функции $g(n)$ при $n > n_0$ с точностью до положительного постоянного множителя, а именно:
 $f(n) = O(g(n))$, если

$$\exists c > 0, n_0 > 0 : \forall n > n_0 \quad 0 \leq f(n) \leq cg(n)$$

$O(g(n))$ обозначает класс функций, таких, что все они растут не быстрее, чем функция $g(n)$ с точностью до положительного постоянного множителя. Говорят, что функция $g(n)$ мажорирует функцию $f(n)$

Paul Bachmann, 1894 > Edmund Landau, 1909 > Ordnung

Оценка O (О большое)



Существуют положительные константы c и n_0 такие, что

$$0 \leq f(n) \leq cg(n)$$

для всех $n \geq n_0$

Оценка O (O большое)

Для функций

$$f_1(n) = 12, \quad f_2(n) = 5n + 23, \quad f_3(n) = n \ln n, \quad f_4(n) = 7n^2 + 12n - 34$$

будет справедливой оценка $O(n^2)$.

Следует указывать наиболее «близкую» мажорирующую функцию. Например, для функции $f(n) = 12n^3$ справедлива оценка $O(2^n)$, но практически она малоприменима

Примеры

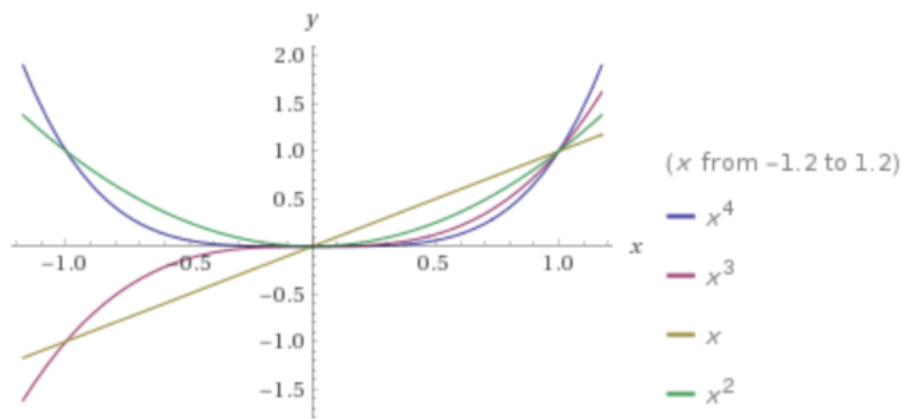
f(n)	g(n)
$2n^2 + 7n - 3$	n^2
$98n \cdot \ln(n)$	$n \cdot \ln(n)$
$5n + 2$	n
8	1

Решим пример

Доказать, что $n^2 + 2n + 1 = O(n^2)$. Найти c, n_0 .

Доказать, что $n^2 + 2n + 1 = O(n^2)$. Найти c, n_0 .

$$n^2 + 2n + 1 \leq c \cdot n^2$$



при $n \geq n_0, n_0 = 1$

$$n^2 \geq n$$

$$n^2 \geq 1$$

$$n^2 + 2n^2 + n^2 \leq C \cdot n^2$$

$$4n^2 \leq Cn^2$$

$$n_0 = 1, C = 4 \Rightarrow \exists c > 0, n_0 > 0: \forall n > n_0 \quad |f(n)| \leq c \cdot |g(n)|$$

ч.т.д.

```
i:=1;
```

```
While (i<=n) And (A[i]<>x) Do
```

```
  I:=i+1;
```

```
If i<=n
```

```
  Then WriteLn( 'Yes' )
```

```
  Else WriteLn( 'No' );
```

Логарифм

Логарифм

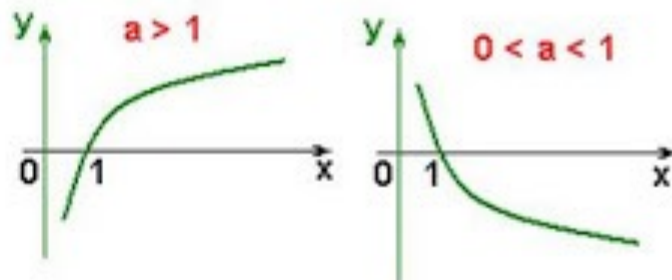
$$\log_a x = b, \text{ если } a^b = x$$

$$\log_a a^b = b$$

$$a^{\log_a x} = x$$

Логарифмическая функция

$$y = \log_a x \quad a > 0, a \neq 1$$



$$\lg x = \log_{10} x$$

$$\ln x = \log_e x$$

$$e = 2,71828...$$

$$e^{\ln x} = x$$

$$e^{x \ln a} = a^x$$

$$\ln e^x = x$$

Свойства

$$1) \log_a x y = \log_a x + \log_a y$$

$$2) \log_a \frac{x}{y} = \log_a x - \log_a y$$

$$3) \log_a x^p = p \cdot \log_a x$$

$$4) \log_a 1 = 0$$

$$5) \log_a a = 1$$

$$6) \log_{a^n} x = \frac{1}{n} \log_a x$$

$$7) \log_{a^n} a^m = \frac{m}{n}$$

$$8) \log_{\left(\frac{1}{a}\right)^n} a^m = -\frac{m}{n}$$

$$9) \log_{a^n} \left(\frac{1}{a}\right)^m = -\frac{m}{n}$$

$$10) \log_b x = \frac{\log_a x}{\log_a b}$$

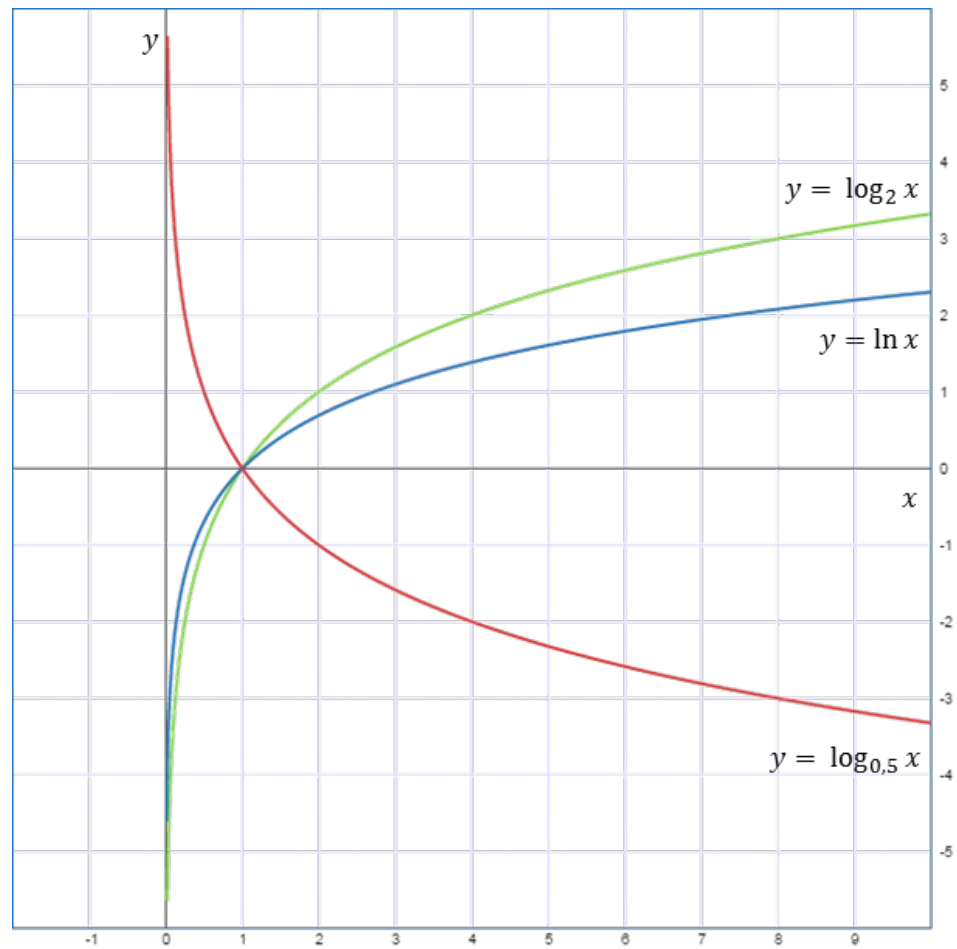
$$\log_2 8 = 3, \text{ так как } 2^3 = 8;$$

$$\log_7 49 = 2, \text{ так как } 7^2 = 49;$$

$$\log_5 \frac{1}{5} = -1, \text{ так как } 5^{-1} = \frac{1}{5};$$

$$\log_3 \sqrt{3} = \frac{1}{2}, \text{ так как } 3^{\frac{1}{2}} = \sqrt{3}.$$

Логарифм



Решим пример

Метод сортировки со сложностью «Big-O» $O(n \log n)$ тратит ровно 1 миллисекунду на сортировку 1000 элементов данных. Предполагая, что время $T(n)$ сортировки n элементов прямо пропорционально $n \log n$, то есть $T(n) = c n \log n$, выведите формулу для $T(n)$, учитывая время $T(N)$ для сортировки N и оцените, как долго этот метод будет сортировать 1 000 000 элементов.

Решим пример

Метод сортировки со сложностью «Big-O» $O(n \log n)$ тратит ровно 1 миллисекунду на сортировку 1000 элементов данных. Предполагая, что время $T(n)$ сортировки n элементов прямо пропорционально $n \log n$, то есть $T(n) = c \cdot n \log n$, выведите формулу для $T(n)$, учитывая время $T(N)$ для сортировки N и оцените, как долго этот метод будет сортировать 1 000 000 элементов.

$$T(N) = c \cdot N \cdot \log N$$

$$T(n) = c \cdot n \cdot \log n \Rightarrow c = \frac{T(n)}{n \cdot \log(n)}$$

$$T(N) = \frac{T(n) \cdot N \cdot \log N}{n \cdot \log n}; \quad T(10^6) = \frac{10^3 \cdot 10^6 \cdot 6}{10^3 \cdot 3} = 2000 \text{ ms}$$

Решим пример

Предположим, что каждое из приведенных ниже выражений дает время обработки $T(n)$, затраченное алгоритмом на решение задачи размера n . Выберите доминирующие члены, имеющие наибольший рост n , и укажите наименьшую сложность Big-Oh для каждого алгоритма.

$5 + 0.001n^3 + 0.025n$		
$500n + 100n^{1.5} + 50n \log_{10} n$		
$0.3n + 5n^{1.5} + 2.5 \cdot n^{1.75}$		
$n^2 \log_2 n + n(\log_2 n)^2$		
$n \log_3 n + n \log_2 n$		
$3 \log_8 n + \log_2 \log_2 \log_2 n$		
$100n + 0.01n^2$		
$0.01n + 100n^2$		
$2n + n^{0.5} + 0.5n^{1.25}$		
$0.01n \log_2 n + n(\log_2 n)^2$		
$100n \log_3 n + n^3 + 100n$		
$0.003 \log_4 n + \log_2 \log_2 n$		

Решим пример

Предположим, что каждое из приведенных ниже выражений дает время обработки $T(n)$, затраченное алгоритмом на решение задачи размера n . Выберите доминирующие члены, имеющие наибольший рост n , и укажите наименьшую сложность Big-Oh для каждого алгоритма.

Expression	Dominant term(s)	$O(\dots)$
$5 + 0.001n^3 + 0.025n$	$0.001n^3$	$O(n^3)$
$500n + 100n^{1.5} + 50n \log_{10} n$	$100n^{1.5}$	$O(n^{1.5})$
$0.3n + 5n^{1.5} + 2.5 \cdot n^{1.75}$	$2.5n^{1.75}$	$O(n^{1.75})$
$n^2 \log_2 n + n(\log_2 n)^2$	$n^2 \log_2 n$	$O(n^2 \log n)$
$n \log_3 n + n \log_2 n$	$n \log_3 n, n \log_2 n$	$O(n \log n)$
$3 \log_8 n + \log_2 \log_2 \log_2 n$	$3 \log_8 n$	$O(\log n)$
$100n + 0.01n^2$	$0.01n^2$	$O(n^2)$
$0.01n + 100n^2$	$100n^2$	$O(n^2)$
$2n + n^{0.5} + 0.5n^{1.25}$	$0.5n^{1.25}$	$O(n^{1.25})$
$0.01n \log_2 n + n(\log_2 n)^2$	$n(\log_2 n)^2$	$O(n(\log n)^2)$
$100n \log_3 n + n^3 + 100n$	n^3	$O(n^3)$
$0.003 \log_4 n + \log_2 \log_2 n$	$0.003 \log_4 n$	$O(\log n)$

Решим пример

Пусть время обработки алгоритма сложности Big-O ($f(n)$) прямо пропорционально $f(n)$. Пусть три таких алгоритма A, B и C имеют временную сложность $O(n^2)$, $O(n^{1.5})$ и $O(n \log n)$ соответственно. Во время теста каждый алгоритм тратит 10 секунд на обработку 100 элементов данных. Найдите время, которое каждый алгоритм потратит на обработку 10000 элементов.

Решим пример

Пусть время обработки алгоритма сложности Big-O ($f(n)$) прямо пропорционально $f(n)$. Пусть три таких алгоритма A, B и C имеют временную сложность $O(n^2)$, $O(n^{1.5})$ и $O(n \log n)$ соответственно. Во время теста каждый алгоритм тратит 10 секунд на обработку 100 элементов данных. Найдите время, которое каждый алгоритм потратит на обработку 10000 элементов.

	Complexity	Time to process 10,000 items
A1	$O(n^2)$	$T(10,000) = T(100) \cdot \frac{10000^2}{100^2} = 10 \cdot 10000 = 100,000 \text{ sec.}$
A2	$O(n^{1.5})$	$T(10,000) = T(100) \cdot \frac{10000^{1.5}}{100^{1.5}} = 10 \cdot 1000 = 10,000 \text{ sec.}$
A3	$O(n \log n)$	$T(10,000) = T(100) \cdot \frac{10000 \log 10000}{100 \log 100} = 10 \cdot 200 = 2,000 \text{ sec.}$

Пример

```
int sum(int n) {  
    if (n == 1) return 1;  
    return n + sum(n - 1);  
}
```

Пример

```
int sum(int n) {  
    if (n == 1) return 1;  
    return n + sum(n - 1);  
}
```

```
int pairSumSequence(int n) {  
    int sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum += pairSum(i, i + 1);  
    }  
    return sum;  
}  
int pairSum(int a, int b) {  
    return a + b;  
}
```


Пример

```
for (int a : arrA) {  
    print(a);  
}  
for (int b : arrB) {  
    print(b);  
}
```

```
for (int a : arrA) {  
    for (int b : arrB) {  
        print(a + "," + b);  
    }  
}
```

Пример

```
void foo(int[] array) {  
    int sum = 0;  
    int product = 1;  
    for (int i = 0; i < array.length; i++)  
        sum += array[i];  
    for (int i = 0; i < array.length; i++)  
        product *= array[i];  
    println(sum + ", " + product);  
}
```

Пример

```
void printPairs(int[] arr) {  
    for (int i = 0; i < arr.length; i++)  
        for (int j = 0; j < arr.length; j++)  
            println(array[i] + "," + array[j]);  
}
```

Пример

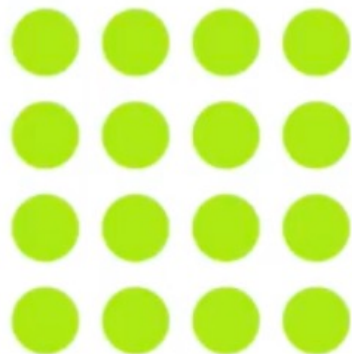
```
for (int i = 0; i < N; i++)  
    for (int j = 0; j < N; j++)  
        foo();
```

```
for (int i = 0; i < N; i++)  
    for (int j = i; j < N; j++)  
        foo();
```

Пример

```
for (int i = 0; i < N; i++)  
  for (int j = 0; j < N; j++)  
    foo();
```

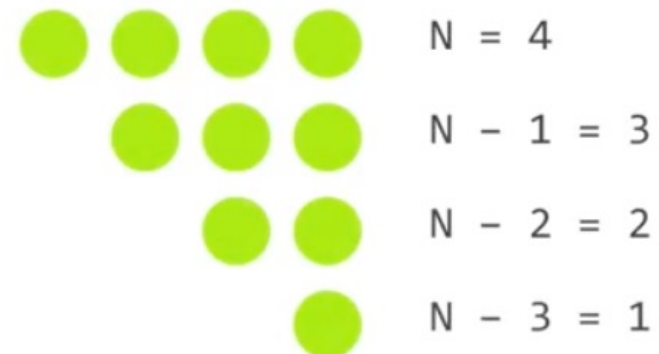
Пусть $N = 4$, тогда:



Сложность выполнения кода
 N^2

```
for (int i = 0; i < N; i++)  
  for (int j = i; j < N; j++)  
    foo();
```

Пусть $N = 4$, тогда:



Сложность выполнения кода
 $\frac{N^2}{2} \approx N^2$

Пример

```
void printPairs(int[] arrA, int[] arrB) {  
    for (int i = 0; i < arrA.length; i++)  
        for (int j = 0; j < arrB.length; j++)  
            if (arrA[i] < arrB[j])  
                println(arrA[i] + "," + arrB[j]);  
}
```

Пример

```
void printPairs(int[] arrA, int[] arrB) {  
    for (int i = 0; i < arrA.length; i++)  
        for (int j = 0; j < arrB.length; j++)  
            for (int k = 0; k < 100000; k++)  
                println(arrA[i] + "," + arrB[j]);  
}
```

Пример

```
void reverse(int[] arr) {  
    for (int i = 0; i < arr.length / 2; i++) {  
        int other = arr.length - i - 1;  
        int temp = arr[i];  
        arr[i] = arr[other];  
        arr[other] = temp;  
    }  
}
```


Поиск в массиве

Search 23

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

23 > 16
take 2nd half

L=0	1	2	3	M=4	5	6	7	8	H=9
2	5	8	12	16	23	38	56	72	91

23 > 56
take 1st half

0	1	2	3	4	L=5	6	M=7	8	H=9
2	5	8	12	16	23	38	56	72	91

Found 23,
Return 5

0	1	2	3	4	L=5 M=5	H=6	7	8	9
2	5	8	12	16	23	38	56	72	91

Array Size	Linear — N	Binary — $\log_2 N$
10	10	4
50	50	6
100	100	7
500	500	9
1000	1000	10
2000	2000	11
3000	3000	12
4000	4000	12
5000	5000	13
6000	6000	13
7000	7000	13
8000	8000	13
9000	9000	14
10000	10000	14

Сложности

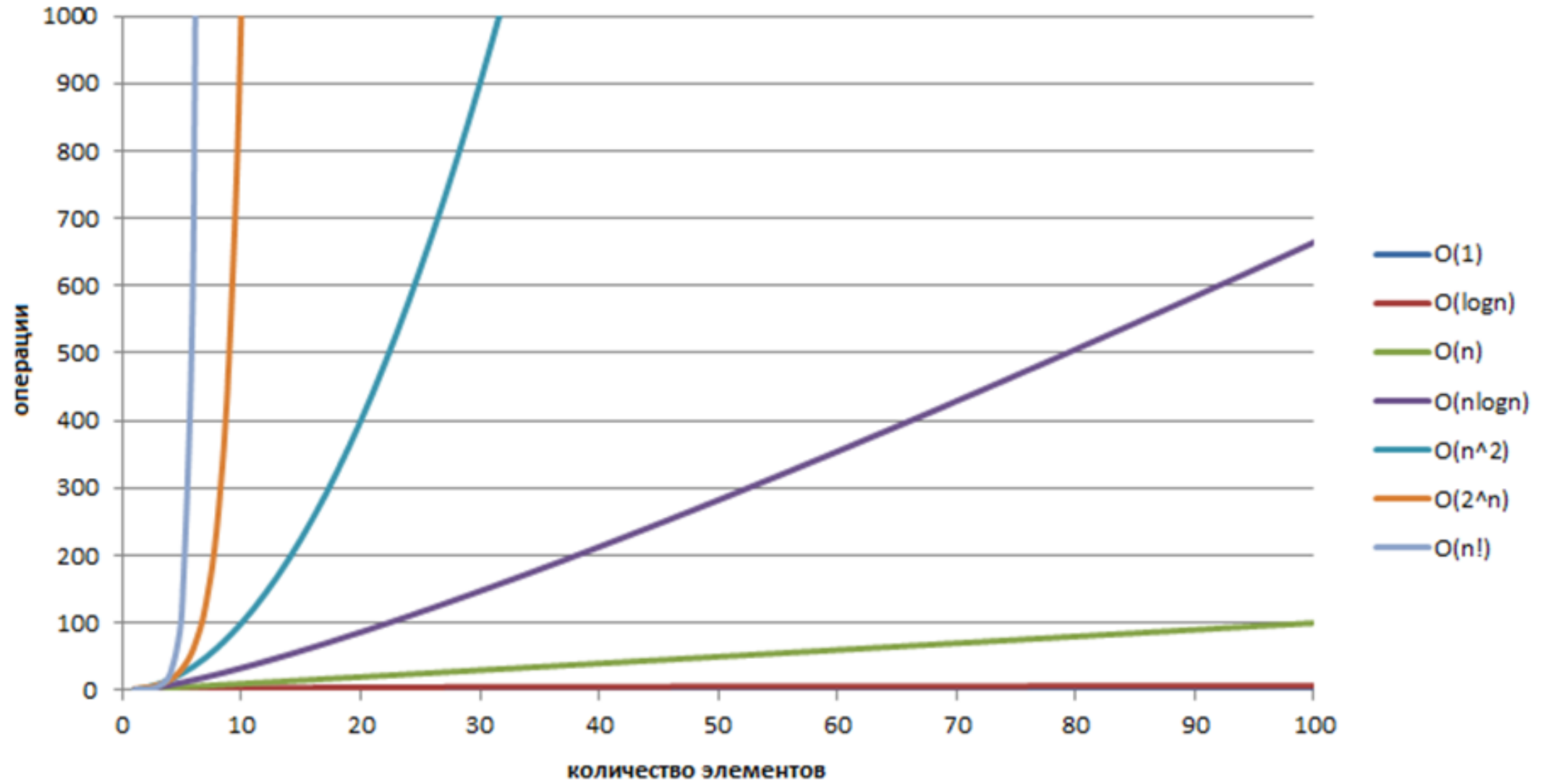
- Постоянная $O(1)$
- Линейная $O(n)$
- Логарифмическая $O(\log n)$
- Квадратичная $O(n^2)$
- Полиномиальная $O(n^k)$
- Экспоненциальная $O(a^n)$
- Факториал $O(n!)$

Что лучше n^2 или $n \log(n)$?

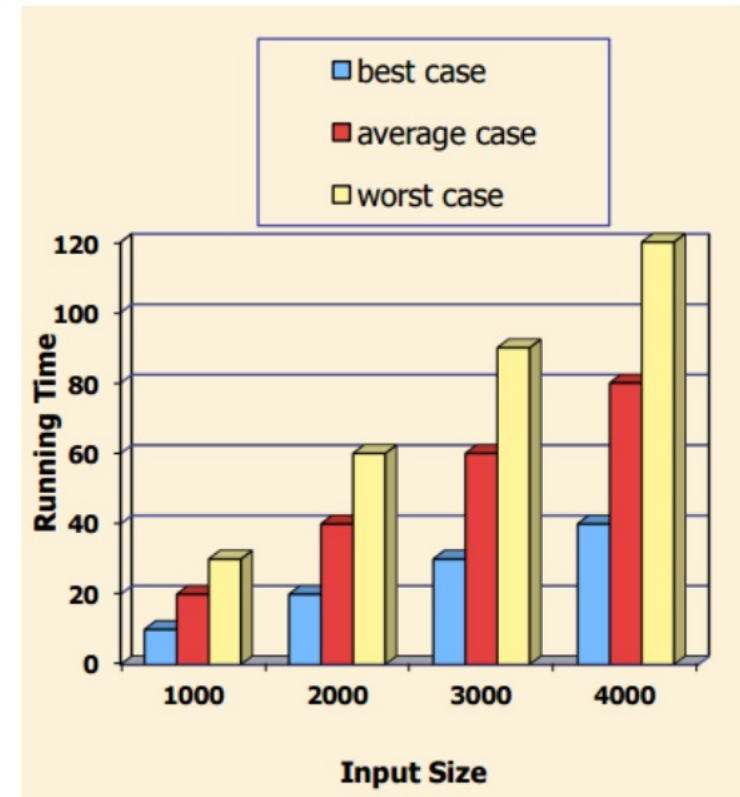
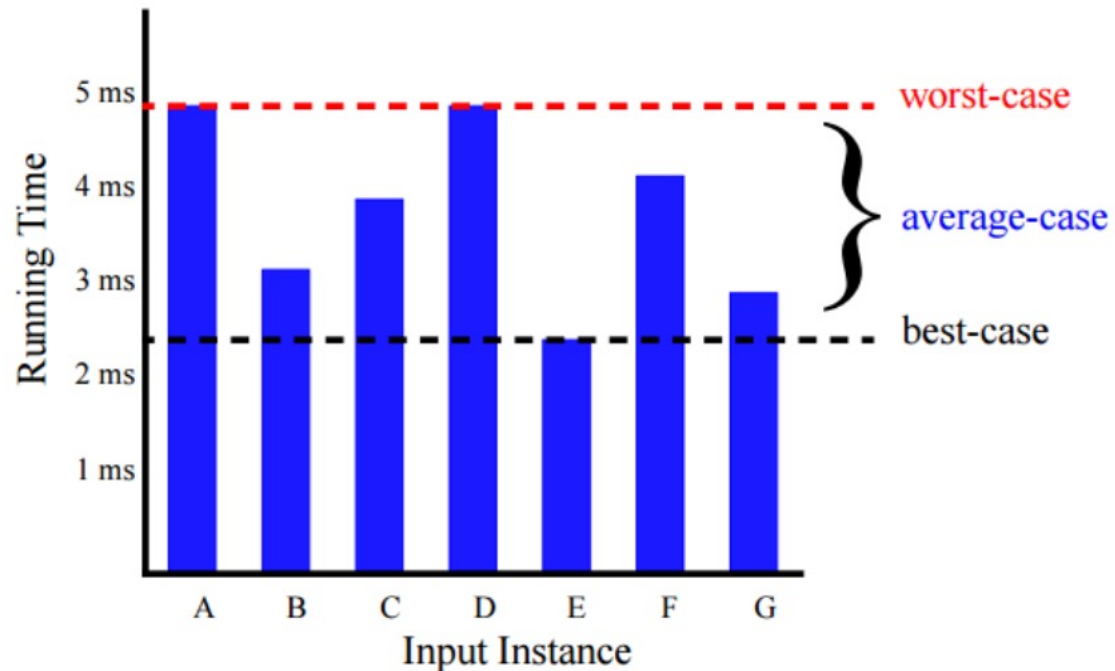
Что лучше n^{100} или 2^n ?

размер сложность	10	20	30	40	50	60
n	0,00001 сек.	0,00002 сек.	0,00003 сек.	0,00004 сек.	0,00005 сек.	0,00005 сек.
n^2	0,0001 сек.	0,0004 сек.	0,0009 сек.	0,0016 сек.	0,0025 сек.	0,0036 сек.
n^3	0,001 сек.	0,008 сек.	0,027 сек.	0,064 сек.	0,125 сек.	0,216 сек.
n^5	0,1 сек.	3,2 сек.	24,3 сек.	1,7 минут	5,2 минут	13 минут
2^n	0,0001 сек.	1 сек.	17,9 минут	12,7 дней	35,7 веков	366 веков
3^n	0,059 сек.	58 минут	6,5 лет	3855 веков	2×10^8 веков	$1,3 \times 10^{13}$ веков

График роста O - большое



Худший, средний и лучший случаи



Пример – последовательный поиск

```
int LinearSearch(int[] arr, int x)
{
    for (int i = 0; i < a.Length; i++)
    {
        if (arr[i] == x)
            return i;
    }
    return -1;
}
```

Алгоритм	Структура данных	Временная сложность			Вспомогательные данные
		Лучшее	В среднем	В худшем	В худшем
Быстрая сортировка	Массив	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$	$O(n)$
Сортировка слиянием	Массив	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Пирамидальная сортировка	Массив	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(1)$
Пузырьковая сортировка	Массив	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Сортировка вставками	Массив	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Сортировка выбором	Массив	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Блочная сортировка	Массив	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(nk)$
Поразрядная сортировка	Массив	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

Классы сложности алгоритмов

Класс P

Проблемы, решение которых считается «быстрым», то есть полиномиально зависящим от размера входных данных (например, $O(n)$, $O(n^2)$)

Примеры

Сортировка

Поиск во множестве

Выяснение связности графов

Класс NP

Проблемы, для решения которых используются лишь алгоритмы, экспоненциально зависящие от размера входных данных (например, $O(2^n)$)

Примеры

Задача коммивояжера

Задача выполнимости булевых формул

Класс ?

Проблемы, для которых сводимость к классу P-проблем и NP-проблем не доказана

Примеры

Факторизация длинных чисел
Дискретное логарифмирование

Оставайтесь любознательными!



Не бойтесь задавать вопросов! 😊